Notes on Kidger's NDEs talks

This is a compilation of (not totally complete) notes on Patrick Kidger's talks across two to three seminars. The aim here is to provide a very high-level overview of subfields within NDEs.

NDEs Overview

A neural differential equation is *a differential equation (DE) that uses a neural network (NN) with learnable parameters/functions to parametrize its vector field*. A canonical starter example is a **neural ordinary differential equation** (NODE):

$$y(0) = y_0 \qquad \frac{dy}{dt}(t) = f_\theta(t, y(t))$$

where $f_\theta : \mathbb{R} \times \mathbb{R}^d \to \mathbb{R}^d$ is some standard neural architecture, oftentimes a feed-forward one. The central idea here is that, instead of treating an ODE solver (e.g. RK4) as an external black-box that just produces numerical solutions, we now embed it inside an NN's differentiable computation graph.

Remark: Traditionally, solvers only integrate and simulate the evolution of DEs with fixed parameters (e.g. of a physical/biological system), without caring how their outputs vary with respect to those parameters. They are used to generate data for an NN, which is trained afterward on these simulated data to approximate the underlying system, making it a separate pipeline from the solver.

▶ More ground-level details about NODE

...

...

It's worth mentioning that this seemingly complicated nesting of an NN inside an DE is nothing groundbreaking from a mathematics or ML point of view alone. For example, the SIR model in epidemiology is as follows:

$$\frac{d}{dt}\begin{pmatrix} s(t) \\ i(t) \\ r(t) \end{pmatrix} = \begin{pmatrix} -bs(t)i(t) \\ bs(t)i(t) - ki(t) \\ ki(t) \end{pmatrix}.$$

Both NODEs and the SIR model are parametrized DEs with some theoretical models for the RHS, with the mere difference being that SIR is specialized to fit a specific phenomenon (namely the evolution of a disease), while an NODE is a generalized "fitter" applicable to lots of things thanks to the NN architecture. In fact, the SIR model can transform into an NODE by making the parameters b and k learnable.

In the framework of deep learning (DL), we may also see a shadow of NODE: by applying Euler method on $\frac{dy}{dt}(t) = f_\theta(t, y(t))$, we obtain a residual network (ResNet):

$$y_{t_{n+1}} = y_{t_n} + \Delta t f_\theta(t_n, y_{t_n}).$$

Interestingly, this connection between DL architectures and NDEs is ubiquitous:

1. ResNet $\leftrightarrow$ neural ordinary differential equations (NODEs)
2. GRUs/LSTMs $\leftrightarrow$ **neural controlled differential equations** (NCDEs)
3. StyleGAN2 $\leftrightarrow$ **neural stochastic differential equations** (NSDEs)

NDEs have a myriad applications in real life, often demonstrating state-of-the-art performance in areas like systematic modelling (physical, biological, financial, etc), regular/irregular time series analysis, generative modelling, and many more.

NDEs offer practically a best-of-both-world approach: the NN side offers high-capacity function approximation and high trainability, while the DE side offers strong prior on model space, memory-efficiency (adjoint backpropagation), and theoretical understanding via well-understood mathematics.

However, NDEs are not without their limitations (speed of computation (might be an implementation issue regarding Python vs C++), number of parameters that insinuates low interpretability and potential overfitting), which we won't discuss much about here.

NCDEs Overview

What is a CDE?

Motivation: consider the integral form of the NODE we saw above:

$$y(0) = y_0 \qquad y(t) = y(0) + \int_0^t f_\theta(y(s))ds$$

which takes the initial condition as the single input. Here, an extra "artificial" time-like dimension is introduced and integrated over, so it's natural to ask if the model can be extended to data of sequential nature (e.g. time series). That is, if we observe some ordered data $(y_0, \ldots, y_n)$ and would like to align this new dimension with the natural ordering of the data, how do we extend the $y(0) = y_0$ condition to one resembling (with the timestamps not necessarily looking like) "$y(0) = y_0, \ldots, y(n) = y_n$"? Since the initial condition at $y(0)$ determines the solution of an ODE, there is no direct mechanism for incorporating the late-coming data. This is where CDEs come into play.

The main idea of a CDE is to have the local dynamics y of the system depend on some time-varying input (path) $X : [0, T] \to \mathbb{R}^v$ in the form of a Riemann-Stieltjes integral

$$y(t) = y(0) + \int_0^t f(y(s))dX(s) \qquad \text{for } t \in [0, T]$$

where $f(y(s))dX(s)$ is a matrix-vector product and can be interpreted as $f(y(s))\frac{dX}{ds}(s)ds$ as long as X is suitably smooth. X can be thought to determine some local notion of physics and fundamentally, a CDE is an operator on path space $X \mapsto y$ that preserves measurability with respect to a filtration (i.e., causality)

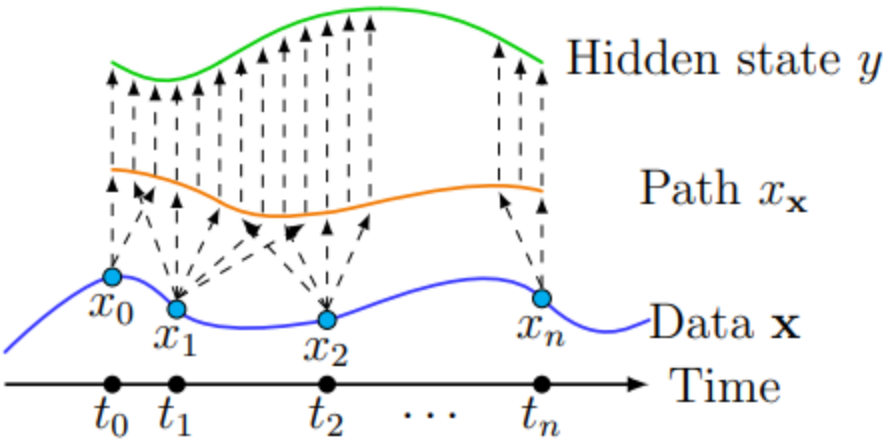
What is an NCDE?

Let $X : [0, T] \to \mathbb{R}^v$ be some observed data, for example an interpolation of a time series, and our goal now is to learn NNs ζ_θ, f_θ, l_θ such that

$$y(0) = \zeta_\theta(X(0)) \qquad y(t) = y(0) + \int_0^t f_\theta(y(s))dX(s)$$

and the output is either $l_\theta(y(T))$ or $l_\theta(y(t))$.

The framework of NCDEs is essentially a continuous version of a recurrent neural network (RNN): $y_{n+1} = f_\theta(y_n, x_n)$, where the discrete inputs x provoke changes in the **hidden state** y, just like how X drive changes continuously in y via the CDE that evolves in the latent space (not the data/output space itself directly, hence the l_θ).



▶ Why do we need this extra latent data?

...

Remark: One might wonder if there's any connection between control theory and NCDEs. Speaking very broadly, the difference between them is somewhat philosophical: the former studies some fixed system f and attempts to find the optimal X producing the desired response y, while the latter has input X fixed and the goal of optimizing f_θ to give the desired y.

There are many things yet to be addressed:

- We know typically f_θ is a feed-forward NN, but how do we initialize f_θ and what are generally some good choices for it?
- What are some interpolation schemes in constructing X? How do we know which one to use?
- The rough path theoretical reasoning behind writing $f_\theta(y(s))dX(s)$ instead of $f_\theta(y(s), X(s))ds$...
- NCDEs are universal approximators for all functional sequences (strong theoretical guarantee): how do we show this and what are its implications?

NSDEs Overview

What is an SDE?

A general SDE has the form

$$dX_t = \mu(t, X_t)dt + \sigma(t, X_t)dW_t$$

where X is some stochastic processes governed by the RHS consisting of drift (μ), diffusion (σ), and a Wiener process/Brownian noise (W_t). Here, X, W, μ are vector-valued, and σ is matrix-valued. The solution to an SDE is a distribution over paths whose sample paths are just "random curves" in time but all following a certain "law" of the underlying process. An SDE solver can sample from an SDE, at least approximately, and is trained by matching statistics with payoff functions F_i:

$$\mathbb{E}_X F_i(X) \approx \mathbb{E}_{data} F_i(data)$$

A quick recap on GANs

Given some noise $A(\omega) \in \mathbb{R}^{d_1}$, target $B(\omega) \in \mathbb{R}^{d_2}$, a *generative model* is an NN $g_\theta : \mathbb{R}^{d_1} \to \mathbb{R}^{d_2}$ such that $g_\theta(A) \approx B$. For example, A may be a multivariate normal sample which, after going through g_θ, produces some distribution resembling B, the distribution of a cat's picture. We can sample $g_\theta(A(\omega))$, but the main difficulty here lies in the lack of tractable probability density, meaning that we can't train g_θ with classical approaches like maximum likelihood. So instead, we train θ to minimize the loss function W defined below, where F_ϕ is the discriminator that is responsible for identifying "fake" data and competing against the GAN's generator:

$$W(g_\theta(A), B) \approx \sup_\phi \left| \frac{1}{N} \sum_{i=1}^{N} F_\phi(g_\theta(A(\omega_i))) - \frac{1}{M} \sum_{i=1}^{M} F_\phi(B(\tilde{\omega}_j)) \right|$$

which is essentially matching statistics: $\mathbb{E}_A F(g_\theta(A)) \approx \mathbb{E}_B F(B)$. The ways that SDE solvers and GANs are trained are very similar, so why not train an SDE as a GAN?

What is an NSDE?

Our goal now is to model B, a random variable on path space (e.g. a distribution of the time series of option prices over the course of 30 days). To now set up the GAN, we take Brownian motion W and initial noise V (e.g. $\sim \mathcal{N}(0,1)$), alongside NNs ζ_θ, μ_θ, σ_θ, l_θ. Then we seek to learn an SDE of the form

$$X_0 = \zeta_\theta(V) \qquad dX_t = \mu_\theta(t, X_t)dt + \sigma_\theta(t, X_t)dW_t \qquad Y_t = l_\theta(X_t)$$

(with a very flexible initial distribution X_0 as it's simply a learnt function of noise V) such that $Y \approx B$. Here, X is the hidden state; Y is the output, which are continuous-time paths, meaning that we need discriminator that accepts such objects as inputs. A convenient choice for F_ϕ would be a CDE (Remember this is an operator between path spaces and we gave an example about classifying handwritten digits? We shall now classify real vs fake time series generated from our SDE using the same CDE framework):

$$H_0 = \xi_\phi(Y_0) \qquad dH_t = f_\phi(t, H_t)dt + g_\phi(t, H_t)dY_t$$

(slightly different form here from using stochastic notations) and then we define $F_\phi := m_\theta \cdot H_T$. Here, m_θ acts as a readout from the hidden state H_T (relative to Y) to the desired discriminator F_ϕ. In short, *NSDE/NCDE form a generator/discriminator pair for a GAN*.

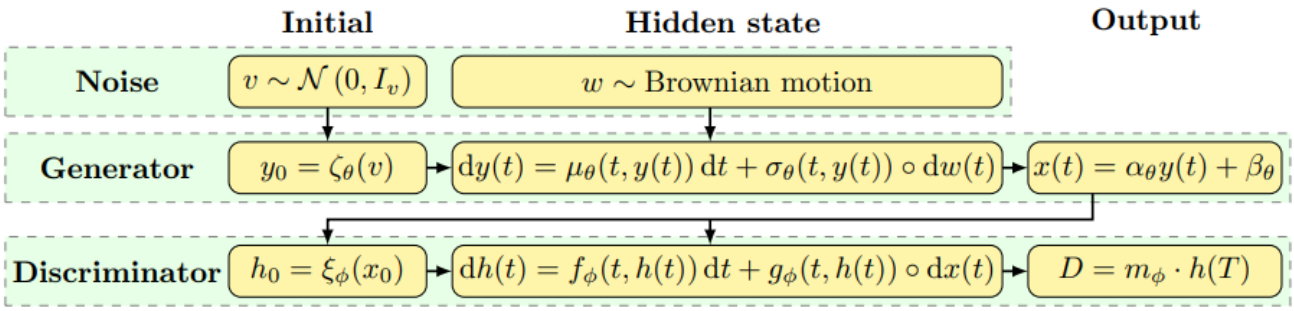
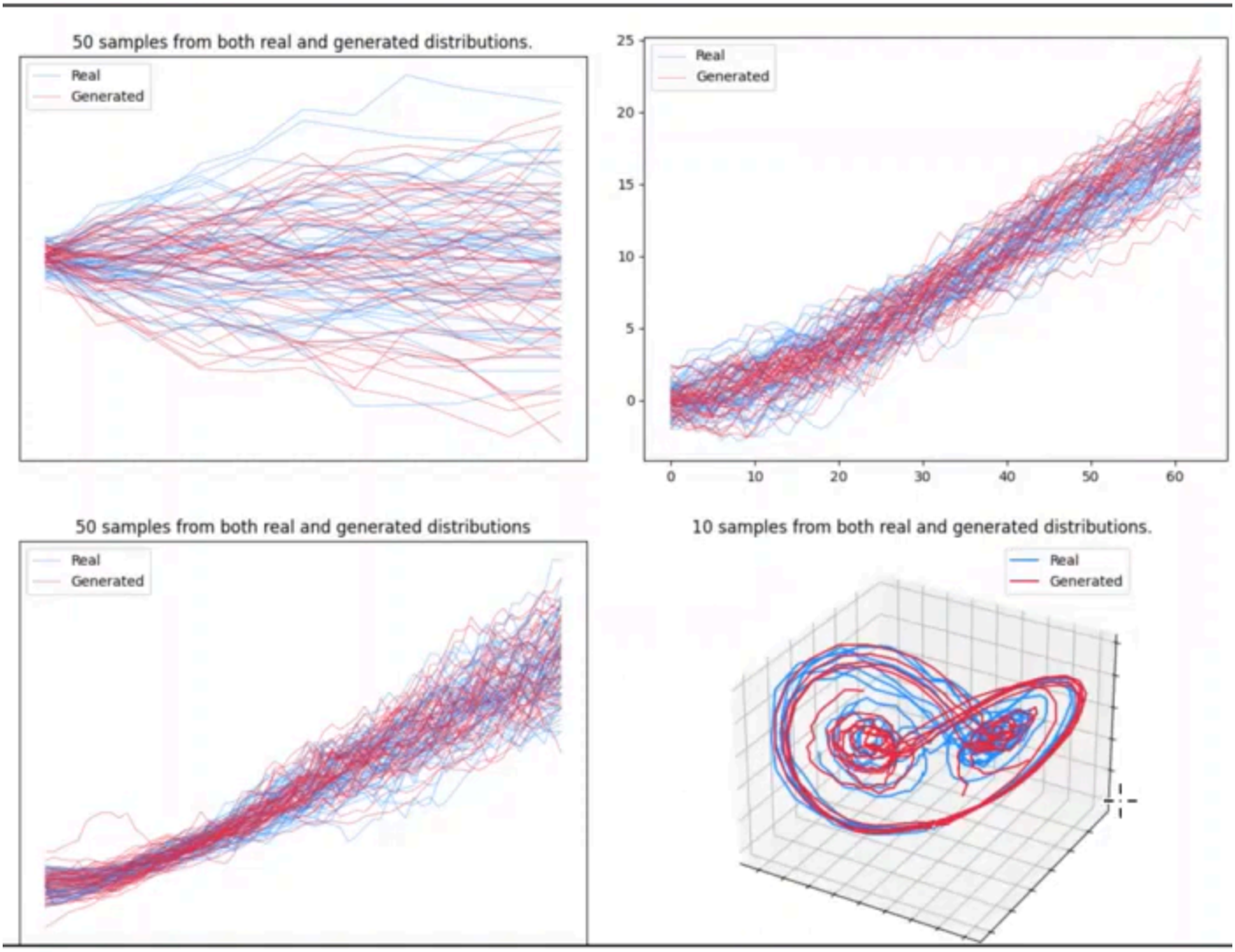


Figure 4.2: Summary of equations for an SDE-GAN.

In Figure 4.2, from top to bottom, it models the formulation of a GAN; from left to right, it models an NDE.

Remark: The only thing required in this framework is those fundamental, off-the-shelf techniques used for a conventional/"non-neural" SDE: not really any new machinery other than making the vector field NNs and learning the f_θ (discriminator of a GAN in this case) instead of using a fixed f.

NSDEs fitting examples (standard Brownian motion with drift or time-dependent diffusion, Lorenz attractor, real-life time series)



Google/Alphabet Stocks:

Metric	Neural SDE	CTFP	Latent ODE
Classification	**0.357 ± 0.045**	0.165 ± 0.087	0.000239 ± 0.000086
Prediction	**0.144 ± 0.045**	0.725 ± 0.233	46.2 ± 12.3
MMD	**1.92 ± 0.09**	2.70 ± 0.47	60.4 ± 35.8

Beijing Air Quality:

Metric	Neural SDE	CTFP	Latent ODE
Classification	0.589 ± 0.051	**0.764 ± 0.064**	0.392 ± 0.011
Prediction	**0.395 ± 0.056**	0.810 ± 0.083	0.456 ± 0.095
MMD	**0.000160 ± 0.000029**	0.00198 ± 0.00001	0.000242 ± 0.000002

SGD dynamics:

Metric	Neural SDE	CTFP	Latent ODE
Classification	0.507 ± 0.019	**0.676 ± 0.014**	0.0112 ± 0.0025
Prediction	**0.00843 ± 0.00759**	0.0808 ± 0.0514	0.127 ± 0.152
MMD	**5.28 ± 1.27**	12.0 ± 0.5	23.2 ± 11.8

In the Beijing Air Quality example (six dimensional markers over the course of a day, time series for several days, modelled using an SDE), one has absolutely *no theory* about how air quality evolves and particulate matter changes over the course of a day, but it doesn't stop us from substituting theory with data, which we fit using NSDEs and other models (uncommon practice in DEs but not in ML) with effective results.